

# Thematic Analysis Codebook and Qualitative Protocol

PA2534 Master's Thesis in Software Engineering

*Decision-Making Under Uncertainty in Bug Triage and Defect Prioritisation*

Naga Sai Abhiram Gopal Vemana

Vineeth Kumar Vajja

Supervisor: Muhammad Laiq

Department of Software Engineering, Blekinge Institute of Technology (BTH), Sweden

May 2026

## Abstract

This document serves as the formal qualitative measurement instrument, coding manual, and audit trail for the thematic analysis conducted on semi-structured practitioner interviews (yielding 2,043 timestamped utterances) in this Master's thesis. Applying Braun and Clarke's (2006) six-phase thematic analysis framework and Lincoln and Guba's (1985) trustworthiness criteria, this codebook defines thirteen operational axial codes organized into three overarching themes: (1) Typology of Triage Uncertainty, (2) Cognitive Heuristics and Strategies under Uncertainty (addressing RQ1), and (3) Organisational and Contextual Constraints (addressing RQ2). The instrument documents explicit inclusion and exclusion rules, inter-coder calibration procedures, negative case analyses, chronological saturation tracking (confirming theoretical saturation at participant  $P_{08}$ ), and mixed-methods triangulation with the parallel cross-sectional survey.

---

## 1 Methodological Framework and Collaborative Coding Protocol

### 1.1 Methodological Grounding

In empirical software engineering research, qualitative analysis must follow transparent, systematic, and auditable procedures [1]. This codebook structures the qualitative data analysis for the thesis by establishing:

1. **Theoretical Foundation:** It applies Braun and Clarke's [2] six-phase thematic analysis method synthesized with empirical software engineering guidelines [1,3].
2. **Shared Conceptual Standard (Investigator Triangulation):** As a collaborative study conducted by two authors, the codebook establishes explicit operational definitions, inclusion boundaries, and disambiguation rules to ensure consistent code application across all transcripts.
3. **Auditable Chain of Evidence:** Following Lincoln and Guba [3], every code is anchored in verbatim, timestamped utterances linked directly to the transcripts dataset.
4. **Mixed-Methods Triangulation:** Every qualitative code is mapped to parallel items in the quantitative survey instrument (SQ1-SQ11), establishing clear data triangulation [4].

### 1.2 Five-Stage Collaborative Coding Workflow

The analysis followed an iterative five-stage protocol conducted jointly by both authors:

- **Stage 1: Independent Open Coding (Calibration Phase):** Both researchers independently performed line-by-line open coding on an initial calibration subset of four diverse transcripts ( $P_{01}$  to  $P_{04}$ ), representing AI architecture, telecom backend, distributed data pipelines, and embedded hardware systems.
- **Stage 2: Negotiated Reconciliation Sessions:** The authors held structured reconciliation meetings to compare applied codes, debate category boundaries, and reach negotiated consensus [2]. Consensus was established through in-depth theoretical debate rather than mechanical statistical coefficients, preserving qualitative nuance.
- **Stage 3: Codebook Formalization:** Thirteen stable axial codes were locked across three thematic dimensions, with explicit inclusion and exclusion criteria.
- **Stage 4: Full Corpus Application:** Both researchers systematically coded the remaining transcripts ( $P_{05}$  to  $P_{12}$ ) using the finalized codebook.
- **Stage 5: Saturation Tracking:** Code emergence was logged sequentially across transcripts  $P_{01}$  through  $P_{12}$ . Zero novel conceptual codes emerged after participant  $P_{08}$ , verifying theoretical saturation across the dataset [5].

### 1.3 Inter-Coder Calibration and Negotiated Consensus Matrix

Table 1 documents the independent coding results, theoretical discussions, and negotiated consensus reached during the calibration phase ( $P_{01}$  to  $P_{04}$ ).

Table 1: Inter-Coder Calibration and Negotiated Consensus Matrix ( $P_{01}$  to  $P_{04}$ )

| Excerpt & Participant                                                                             | Researcher 1 (Abhiram)                   | Researcher 2 (Vineeth)                             | Theoretical Debate & Resolution                                                                                          | Consensus Code                                           |
|---------------------------------------------------------------------------------------------------|------------------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <b>Segment A</b> ( $P_{01}$ )<br>Missing inputs; querying Athena, Jira, PowerBI.                  | Incomplete Reports & Tool Search         | Defect Incompleteness ( <i>Pair</i> : Tool Search) | R1 grouped tool search into incompleteness. R2 argued that the information gap must be decoupled from the search action. | UNC-INCOMP<br>(HEUR-MIN-DELTA)                           |
| <b>Segment B</b> ( $P_{02}$ )<br>5 years in fault management; pattern matching; lead talk.        | Experience Pattern Matching              | Pattern Recognition ( <i>Pair</i> : Lead Triage)   | Strong agreement on Klein’s RPD model. Shift from individual cognition to team consultation split into two codes.        | HEUR-HIST-ANALOG<br>→<br>ORG-SOC-CONSULT                 |
| <b>Segment C</b> ( $P_{03}$ )<br>QA marks blocker on edge case; devs argue under sprint deadline. | Severity Conflict Under Sprint Deadlines | Inter-Role Severity Discrepancy                    | Both agreed on severity conflict. R1 stressed conflict; R2 emphasized that sprint deadlines triggered the escalation.    | UNC-SEV-AMBIG<br>( <i>Driven by</i> ORG-RELEASE-GATE)    |
| <b>Segment D</b> ( $P_{04}$ )<br>Intermittent telecom bug; release blocker requires hardware fix. | Non-Repro Bug & Mandatory Fix            | Non-Deterministic Fault in Hardware                | Both identified non-reproducibility. Crucially, both identified this as an explicit negative case refuting workarounds.  | UNC-NONREPRO<br>( <i>Negative Case</i> : HEUR-PRAG-WORK) |

## 1.4 Operational Coding Decision Rules

Three universal coding rules were derived from the calibration phase:

1. **Decision Rule 1 (Separation of Problem from Action):** An epistemic information deficit in a ticket is coded under Theme 1 (UNC-\*). The human cognitive coping action is coded under Theme 2 (HEUR-\*).
2. **Decision Rule 2 (Multi-Clause Splitting):** Complex turns containing multiple cognitive moves (e.g., historical pattern recognition followed by peer consultation) are split at clausal boundaries and coded independently.
3. **Decision Rule 3 (Contextual Moderation):** Severity disagreements are coded as UNC-SEV-AMBIG; if the discussion indicates the dispute was triggered by an impending release milestone, ORG-RELEASE-GATE is double-tagged.

## 2 Master Thematic Prevalence and Mixed-Methods Triangulation

### 2.1 Qualitative Prevalence Calculation

Following Braun and Clarke [2] and empirical software engineering guidelines [1], the qualitative prevalence of each code is calculated as the participant-level occurrence rate across the interview corpus:

$$\text{Qualitative Prevalence (\%)} = \left( \frac{n_{\text{participants}}}{N_{\text{total}}} \right) \times 100\% = \left( \frac{n}{12} \right) \times 100\% \quad (1)$$

where  $n_{\text{participants}}$  denotes the number of distinct interview participants whose transcripts contain at least one verified instance of the code, and  $N_{\text{total}} = 12$ .

Table 2 reports the verified qualitative distribution, prevalence rates, and direct triangulation against responses from the cross-sectional practitioner survey.

### 2.2 Mixed-Methods Triangulation Synthesis

#### 1. Direct Qualitative-Quantitative Convergence:

- *Incomplete Descriptions (UNC-INCOMP):* Universal in interviews (100.0%), mirrored in the survey where 47.7% ( $n = 31$ ) ranked ticket comprehension as their greatest bottleneck (SQ4) and 40.0% ( $n = 26$ ) cited missing steps (SQ6).
- *Asynchronous Information Lag (UNC-ASYNC-LAG):* 66.7% of interviewees described ticket idle time while awaiting reporter replies; 66.2% ( $n = 43$ ) of surveyed engineers confirmed entering immediate wait states upon receiving ambiguous tickets (SQ7).
- *Release Deadline Pressure (ORG-RELEASE-GATE):* 75.0% of interviewees noted that release proximity enforces binary release-blocking assessments; 44.6% ( $n = 29$ ) of survey respondents ranked release schedules as their primary constraint (SQ9).

2. **Seniority Moderation on Blast-Radius Assessment (HEUR-BLAST-RAD):** While 83.3% of interviewees (primarily senior leads and architects) emphasized blast-radius evaluation, only 18.5% ( $n = 12$ ) of the general survey sample selected it as their default heuristic (SQ8). Cross-tabulating SQ8 by experience reveals that blast-radius evaluation surges to **42.9% among senior practitioners (> 10 years experience)**, while junior engineers (< 1 year) predominantly rely on administrative default medium ratings (40.0%). This confirms Klein’s [6] RPD model: blast-radius assessment is an **expertise-driven heuristic**.

Table 2: Master Thematic Prevalence and Mixed-Methods Triangulation Matrix

| Code ID                                                                   | Operational Descriptor               | Qual. Distribution                    | Prevalence (Count) | Survey Triangulation                                  |
|---------------------------------------------------------------------------|--------------------------------------|---------------------------------------|--------------------|-------------------------------------------------------|
| <b>Theme 1: Typology of Triage Uncertainty (Contextual Baseline)</b>      |                                      |                                       |                    |                                                       |
| UNC-INCOMP                                                                | Incomplete Descriptions & Steps      | P01 to P12 (All)                      | 100.0% (12/12)     | 47.7% ( $n = 31$ ) [SQ4]<br>40.0% ( $n = 26$ ) [SQ6]  |
| UNC-NONREPRO                                                              | Environmental Divergence & Flakiness | P02, P03, P04, P06 to P12             | 83.3% (10/12)      | 41.5% ( $n = 27$ ) [SQ4]<br>55.4% ( $n = 36$ ) [SQ6]  |
| UNC-ASYNC-LAG                                                             | Asynchronous Information Lag         | P01, P03, P05, P07 to P11             | 66.7% (8/12)       | 66.2% ( $n = 43$ ) [Wait Clarification SQ7]           |
| UNC-SEV-AMBIG                                                             | Severity Ambiguity & Disagreement    | P01, P03, P05, P06, P08, P10 to P12   | 66.7% (8/12)       | 18.5% ( $n = 12$ ) [Cross-Team Conflict SQ9]          |
| <b>Theme 2: Cognitive Heuristics and Strategies (Addressing RQ1 / O2)</b> |                                      |                                       |                    |                                                       |
| HEUR-BLAST-RAD                                                            | Blast-Radius Prioritization          | P01 to P03, P05, P07 to P12           | 83.3% (10/12)      | 18.5% gen. → <b>42.9% in senior (&gt; 10yr) [SQ8]</b> |
| HEUR-HIST-ANAL                                                            | Historical Pattern Matching          | P01, P02, P04, P07 to P09, P11, P12   | 66.7% (8/12)       | 38.5% ( $n = 25$ ) [Pattern Match SQ8]                |
| HEUR-TIME-BOX                                                             | Investigation Time-Boxing            | P01, P03, P05, P08, P10 to P12        | 58.3% (7/12)       | 27.7% ( $n = 18$ ) [Administrative Deferral SQ8]      |
| HEUR-MIN-DELTA                                                            | Minimum Delta Probing                | P01, P02, P04, P06, P07, P09 to P12   | 75.0% (9/12)       | 23.1% ( $n = 15$ ) [Repo/Diff Search SQ7]             |
| HEUR-PRAG-WORK                                                            | Decoupled Pragmatic Mitigation       | P01, P03, P05, P08, P10, P11          | 50.0% (6/12)       | Qualitative Survey Text (Resp_14, Resp_48)            |
| <b>Theme 3: Organisational Constraints (Addressing RQ2 / O3)</b>          |                                      |                                       |                    |                                                       |
| ORG-RELEASE-GATE                                                          | Release Gating & Deadlines           | P01 to P03, P05, P07, P08, P10 to P12 | 75.0% (9/12)       | 44.6% ( $n = 29$ ) [Ranked #1 Constraint SQ9]         |
| ORG-MGMT-ALIGN                                                            | Management & Business SLAs           | P01, P03, P06, P08, P11, P12          | 50.0% (6/12)       | 23.1% ( $n = 15$ ) [Commercial SLAs SQ9]              |
| ORG-LOAD-THROT                                                            | Workload Throttling & Backlog Load   | P01, P05, P07, P08, P10 to P12        | 58.3% (7/12)       | 40.0% ( $n = 26$ ) [Backlog Scale SQ9]                |
| ORG-SOC-CONSULT                                                           | Social Cross-Team Consensus          | P02, P04, P06, P08, P11, P12          | 50.0% (6/12)       | 18.5% ( $n = 12$ ) [Synchronous Escalation SQ9]       |

## 3 Detailed Operational Code Specifications

### 3.1 Theme 1: Typology of Triage Uncertainty (Contextual Baseline)

#### 3.1.1 UNC-INCOMP: Incomplete Descriptions and Missing Steps

- **Definition:** Defect reports that lack critical diagnostic information: reproducible steps, input data payloads, expected vs. observed outcomes, or system version details.
- **Theoretical Grounding:** Epistemic Uncertainty [7]: uncertainty arising from incomplete information.
- **Inclusion Criteria:** Apply when reports lack steps, input parameters, logs, or error traces; apply when triagers describe having to query secondary monitoring systems to reconstruct what happened.
- **Exclusion Criteria:** Do **NOT** apply if parameters are complete but local reproduction fails due to environment differences (Code as UNC-NONREPRO).
- **Anchor Quotation:** “When a bug report is incomplete, it might not contain the exact input data, timestamps, or system version... understanding the issue requires looking at Athena views, Jira tickets, and PowerBI dashboards to reconstruct what happened.” ( $P_{01}$ , AI Product Architect)
- **Survey Triangulation:** 47.7% ranked description comprehension as the 1 bottleneck (SQ4); 40.0% cited missing reproduction steps (SQ6).

#### 3.1.2 UNC-NONREPRO: Environmental Divergence and Intermittent Bugs

- **Definition:** Defects that manifest in staging, client clusters, or production environments but cannot be deterministically reproduced locally due to environmental drift, concurrency races, or stateful caching anomalies.
- **Theoretical Grounding:** Aleatoric Uncertainty and System Stochasticity.
- **Inclusion Criteria:** Apply when participants describe “works on my machine” syndromes, flaky tests, non-deterministic bugs occurring 1 in 10 or 1 in 50 runs, or OS/Kubernetes cluster discrepancies.
- **Exclusion Criteria:** Do **NOT** apply if the bug fails to reproduce simply because steps were omitted (Code as UNC-INCOMP).
- **Anchor Quotation:** “In an industry like telecom where you work across modules, most of the time there could be intermittent issues... which is flaky intermittent not reproducible... We ask: is this consistently reproducible? Is it 1 in 10 runs, 1 in 50 runs? Those are the details that are normally missing.” ( $P_{04}$ , Senior Embedded Technical Lead)
- **Survey Triangulation:** 41.5% ranked local reproduction as primary bottleneck (SQ4); 55.4% cited environment discrepancies (SQ6).

#### 3.1.3 UNC-ASYNC-LAG: Information Latency and Feedback Loops

- **Definition:** Ticket idle time and communication delays introduced when requesting clarification across distributed teams or multi-tiered support channels.
- **Theoretical Grounding:** Asynchronous Coordination Latency [8].
- **Inclusion Criteria:** Apply when participants describe delays experienced after asking reporters for logs, waiting overnight across time zones, or managing ticket ping-pong.

- **Exclusion Criteria:** Do **NOT** apply if engineers bypass ticketing via an instant voice call or Slack huddle (Code as **ORG-SOC-CONSULT**).
- **Anchor Quotation:** “My first step is to identify the smallest amount of missing information that could explain the difference... this is more efficient than asking for a large amount of information without knowing what is actually missing, which drags out communication.” ( $P_{05}$ , Full-Stack Software Engineer)
- **Survey Triangulation:** 66.2% ( $n = 43$ ) stated their immediate first action is requesting details from the reporter and entering a wait state (SQ7).

### 3.1.4 UNC-SEV-AMBIG: Severity Ambiguity and Stakeholder Disagreements

- **Definition:** Conflicting evaluations between QA, developers, product managers, and operations regarding the true priority and urgency of a defect.
- **Theoretical Grounding:** Conflict Uncertainty [7]: divergent mental models among organizational stakeholders.
- **Inclusion Criteria:** Apply when QA rates an issue as a blocker but developers view it as an ignorable edge case, or vice versa; apply when boundary between bug and feature request is contested.
- **Exclusion Criteria:** Do **NOT** apply if priority is decided strictly by a contractual SLA or executive mandate (Code as **ORG-MGMT-ALIGN**).
- **Anchor Quotation:** “Sometimes the developers and operations teams may have different opinions about priority in that situation... we balance technical impact, customer impact, business importance, and available resources.” ( $P_{11}$ , DevOps & Cloud Platform Engineer)
- **Survey Triangulation:** 18.5% ( $n = 12$ ) reported cross-team priority disagreements as a key operational constraint (SQ9).

## 3.2 Theme 2: Cognitive Heuristics and Strategies Under Uncertainty (RQ1 / O2)

### 3.2.1 HEUR-BLAST-RAD: Blast-Radius and Critical Path Prioritization

- **Definition:** Prioritizing defects by evaluating the potential scope of system damage (affected user volume, financial/payment pipelines, authentication endpoints, or security integrity) before root cause isolation.
- **Theoretical Grounding:** Klein’s Recognition-Primed Decision (RPD) Model [6].
- **Inclusion Criteria:** Apply when engineers assess systemic impact (e.g., customer-facing status, payment corruption, authentication failure) to assign priority under incomplete reproduction data.
- **Exclusion Criteria:** Do **NOT** apply when priority is escalated solely because of a client complaint without technical assessment (Code as **ORG-MGMT-ALIGN**).
- **Anchor Quotation:** “Well, an issue blocking a production deployment would obviously have higher priority... If an issue affects customer-facing service, it immediately gets highest priority even if the exact root cause is still uncertain.” ( $P_{11}$ , DevOps & Cloud Platform Engineer)
- **Survey Triangulation:** 18.5% general survey sample → **42.9% among senior engineers (> 10 years experience)** [SQ8].

### 3.2.2 HEUR-HIST-ANALOG: Historical Analogy and Pattern Matching

- **Definition:** Diagnosing ambiguous symptoms by matching current error signatures against accumulated experience, historical post-mortems, or prior bug reports.
- **Theoretical Grounding:** Klein’s RPD Prototype Recognition [6].
- **Inclusion Criteria:** Apply when experience enables immediate recognition of which module is failing based on past outages or similar test failures; apply when searching Jira/Git for historical duplicates.
- **Exclusion Criteria:** Do **NOT** apply when engineers systematically partition the search space via commit diff bisection (Code as HEUR-MIN-DELTA).
- **Anchor Quotation:** “From past five years of experience in telecom fault management, when a call comes in production, you recognize the pattern from previous test failures and know where to look.” ( $P_{02}$ , Backend Developer, Telecom)
- **Survey Triangulation:** 38.5% ( $n = 25$ ) selected historical pattern matching as their primary rule of thumb under uncertainty (SQ8).

### 3.2.3 HEUR-TIME-BOX: Investigation Time-Boxing and Cognitive Budgeting

- **Definition:** Allocating a bounded temporal budget (e.g., 30 minutes to 2 hours) to attempt reproduction; if unresolved when the budget expires, deep investigation is suspended and a default priority is assigned.
- **Theoretical Grounding:** Bounded Rationality and Satisficing [7].
- **Inclusion Criteria:** Apply when participants explicitly mention allocating a set time-box for initial investigation before moving on or escalating; apply when assigning a default priority (e.g., medium) to defer investigation.
- **Exclusion Criteria:** Do **NOT** apply when work is halted specifically by a deployment milestone freeze (Code as ORG-RELEASE-GATE).
- **Anchor Quotation:** “Workload definitely affects investigation depth... In that situation, we need to time-box the initial investigation... is the issue real? Is it reproducible? And does it affect critical paths? If unresolved, we move it down.” ( $P_{10}$ , Full-Stack Developer)
- **Survey Triangulation:** 27.7% ( $n = 18$ ) assign a default medium priority when technical certainty is unachievable within reasonable effort (SQ8).

### 3.2.4 HEUR-MIN-DELTA: Minimum Delta Probing and Architectural Bisection

- **Definition:** Narrowing the diagnostic search space by isolating the smallest possible variable difference: partitioning the problem into Data vs. Config vs. Code, bisecting Git commit histories, or diffing environment manifests.
- **Theoretical Grounding:** Fault Localization Bisection.
- **Inclusion Criteria:** Apply when partitioning the problem into categories (data vs. config vs. code); apply when comparing environment diffs (Kubernetes manifests, package versions, Kafka offsets).
- **Exclusion Criteria:** Do **NOT** apply when implementing temporary workarounds (Code as HEUR-PRAG-WORK).

- **Anchor Quotation:** “And the moment a new report arrives, we check whether most of our issues fall into data, or if it would be a configuration, or it would be a code issue... to narrow down the variable space.” ( $P_{12}$ , Backend Software Engineer)
- **Survey Triangulation:** 23.1% ( $n = 15$ ) stated their first action is searching repositories, commit logs, and configuration diffs (SQ7).

### 3.2.5 HEUR-PRAG-WORK: Decoupled Pragmatic Mitigation (Safeguards vs. Cause)

- **Definition:** Separating immediate operational risk mitigation (feature toggles, defensive null checks, increased timeouts, circuit breakers) from permanent root-cause identification, allowing delivery to proceed.
- **Theoretical Grounding:** Risk Containment and Pragmatic Satisficing.
- **Inclusion Criteria:** Apply when engineers unblock production using temporary workarounds, retry limits, or configuration rollbacks while keeping the bug ticket open in the backlog.
- **Exclusion Criteria:** Do **NOT** apply when a developer fixes the true underlying bug in code immediately.
- **Anchor Quotation:** “Normally, I would spend more time doing a deeper root cause analysis... So we implemented additional time-out handling, improved logging and added monitoring... under deadline pressure, sometimes we separate immediate mitigation from permanent root cause.” ( $P_{10}$ , Full-Stack Developer)
- **Survey Triangulation:** Corroborated in open-ended survey comments emphasizing that stabilizing user experience via feature flags precedes deep debugging (Resp\_14, Resp\_48).

## 3.3 Theme 3: Organisational and Contextual Constraints (RQ2 / O3)

### 3.3.1 ORG-RELEASE-GATE: Release Gating and Deadline Compression

- **Definition:** Impending deployment milestones, sprint freezes, and scheduled release cutoffs that compress analytical investigation time, forcing triagers to adopt binary release-blocking evaluation criteria.
- **Theoretical Grounding:** Temporal Compression [9]: deadline proximity shifting decision thresholds from thorough analysis to risk-taking satisficing.
- **Inclusion Criteria:** Apply when triage criteria convert into a strict binary question: “Does this block deployment, yes or no?”; apply when non-critical bugs are systematically deferred to the next release.
- **Exclusion Criteria:** Do **NOT** apply to personal time-boxing chosen during routine non-deadline work (Code as HEUR-TIME-BOX).
- **Anchor Quotation:** “Yes, release deadlines definitely affect how we handle uncertain bugs... when the deadline is close, we cannot spend days investigating. We evaluate whether the bug blocks deployment; if not, we postpone the deep investigation.” ( $P_{11}$ , DevOps & Cloud Platform Engineer)
- **Survey Triangulation:** Ranked as the 1 organizational constraint by 44.6% ( $n = 29$ ) of surveyed practitioners (SQ9).

### 3.3.2 ORG-MGMT-ALIGN: Management Expectations and Business SLAs

- **Definition:** Commercial priorities, executive escalations, enterprise customer contracts, and SLA penalty risks overriding purely technical severity metrics during defect prioritization.
- **Theoretical Grounding:** Organizational Alignment: technical defect evaluation operating in response to broader business commitments.
- **Inclusion Criteria:** Apply when a technically low-severity bug is promoted because it affects a VIP client or enterprise account; apply when product management dictates priority over developer assessments.
- **Exclusion Criteria:** Do **NOT** apply when priority is based strictly on objective technical blast-radius (Code as HEUR-BLAST-RAD).
- **Anchor Quotation:** “And there might be several bugs with similar technical severity, but if one affects customer facing services or a major client, management expectations factor into the triage decision... we balance technical impact, customer impact, business importance and available resources.” ( $P_{11}$ , DevOps & Cloud Platform Engineer)
- **Survey Triangulation:** 23.1% ( $n = 15$ ) identified management expectations and commercial business commitments as a primary constraint (SQ9).

### 3.3.3 ORG-LOAD-THROT: Workload Throttling and Cognitive Overload Management

- **Definition:** Defensive filtering and triage-by-exception strategies enacted by engineers to prevent cognitive exhaustion when confronted with massive ticket backlogs or noisy alert streams.
- **Theoretical Grounding:** Information Overload Filtering [10]: rationing mental resources by aggressively pruning low-probability defects.
- **Inclusion Criteria:** Apply when handling hundreds of open issues, noisy monitoring systems, or massive backlogs where investigating everything with equal depth is impossible.
- **Exclusion Criteria:** Do **NOT** apply when setting a personal 2-hour timer on a single complex ticket (Code as HEUR-TIME-BOX).
- **Anchor Quotation:** “Once an application goes into QA or production, defects start coming from the testing team, monitoring systems, clients and other developers... you cannot investigate everything with the same depth. You check severity, reduce scope, and decide whether it needs an immediate fix or slides into the stack.” ( $P_{10}$ , Full-Stack Developer)
- **Survey Triangulation:** Backlog scale and workload volume was cited by 40.0% ( $n = 26$ ) as a decisive constraint (SQ9).

### 3.3.4 ORG-SOC-CONSULT: Social Consultation and Cross-Functional Consensus

- **Definition:** Bypassing formal asynchronous ticketing systems to resolve ambiguous defects through synchronous, informal communication channels: ad-hoc Slack/Teams huddles, standup discussions, or architectural consultations.
- **Theoretical Grounding:** Transactive Memory Systems and Collaborative Decision-Making [8].
- **Inclusion Criteria:** Apply when participants describe jumping onto video calls, messaging module leads directly, or holding joint troubleshooting sessions to reach consensus.

- **Exclusion Criteria:** Do **NOT** apply when triagers send a ticket comment and passively await a response (Code as UNC-ASYNC-LAG).
- **Anchor Quotation:** “Where we will be working with multiple modules across... one module might be doing end-to-end testing, module testing, and multiple modules also can be tested. In that case, we will get some bugs from various teams or the testers.” ( $P_{04}$ , Senior Embedded Technical Lead)
- **Survey Triangulation:** 18.5% ( $n = 12$ ) cited cross-team disagreements as an organizational challenge (SQ9); qualitative accounts show engineers resolve these via synchronous social consultation.

## 4 Methodological Rigor and Negative Case Analysis

### 4.1 Adherence to Lincoln and Guba’s Trustworthiness Criteria

#### 1. Credibility (Internal Validity):

- *Incident-Anchored Inquiries:* Questions were anchored to specific, recent defect episodes rather than abstract routines, preventing retrospective rationalization [11].
- *Member Checking:* Thematic structures were verified with practitioner feedback [3].

#### 2. Transferability (External Validity):

- *Thick Description:* Contextual demographic and tooling profiles are documented for all 12 interviewees spanning telecom, AI, cloud SaaS, and embedded systems [1].
- *Survey Validation:* Inductive findings were cross-checked against a cross-sectional practitioner survey sample across diverse industry backgrounds.

#### 3. Dependability (Reliability):

- *Auditable Codebook:* Explicit, rule-based inclusion and exclusion criteria ensure that independent researchers can replicate coding decisions across the dataset.
- *Automated Transcription Pipeline:* Audio files were processed using an auditable, deterministic Faster-Whisper pipeline with 100% segment parity verification.

#### 4. Confirmability (Objectivity):

- *Investigator Triangulation:* Coding was conducted collaboratively by both researchers with structured reconciliation sessions to resolve interpretative variance [2].
- *Verbatim Grounding:* Every code is supported by timestamped verbatim quotations verified in the Transcript Fidelity Audit Report.

### 4.2 Negative Case Analysis: Safety-Critical Firmware vs. Cloud Workarounds

A critical mark of qualitative research maturity is documenting empirical negative cases that qualify an overarching pattern [3]. While cloud-native developers ( $P_{10}, P_{11}$ ) regularly enact decoupled pragmatic workarounds (‘HEUR-PRAG-WORK’) to unblock production releases, Participant  $P_{04}$  (Senior Embedded Lead with 20+ years in telecom) presented a clear counter-example:

“Coming to the software that we build, we build the embedded software which runs on specific hardware, proprietary hardware in telecom... Any bug that’s been reported, they normally keep as medium until unless the software can’t be deployable in that case they might go as critical... If any critical bugs which can block the release, that would take highest priority and that has to be handled before the release happens...”

otherwise we cannot do the release, so priorities will increase and we actually need to fix the bugs before they deploy.” ( $P_{04}$ )

This negative case establishes a fundamental boundary condition: **heuristic time-boxing and decoupled mitigation are bounded by system criticality and domain governance**. In safety-critical and hardware-coupled systems, regulatory and physical constraints mandate exhaustive root-cause resolution over heuristic satisficing.

## 5 Code Emergence and Theoretical Saturation Progression Matrix

Table 3 logs the chronological emergence of each code across the 12 interview transcripts ( $P_{01}$  through  $P_{12}$ ), demonstrating empirical saturation.

Table 3: Chronological Code Emergence and Theoretical Saturation Matrix ( $P_{01}$  to  $P_{12}$ )

| Code ID          | $P_{01}$ | $P_{02}$ | $P_{03}$ | $P_{04}$ | $P_{05}$ | $P_{06}$ | $P_{07}$ | $P_{08}$ | $P_{09}$ | $P_{10}$ | $P_{11}$ | $P_{12}$ | Emergence Horizon      |
|------------------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|------------------------|
| UNC-INCOMP       | ★        | ●        | ●        | ●        | ●        | ●        | ●        | ●        | ●        | ●        | ●        | ●        | Discovered in $P_{01}$ |
| UNC-NONREPRO     | -        | ★        | ●        | ●        | -        | ●        | ●        | ●        | ●        | ●        | ●        | ●        | Discovered in $P_{02}$ |
| UNC-ASYNC-LAG    | ★        | -        | ●        | -        | ●        | -        | ●        | ●        | ●        | ●        | ●        | -        | Discovered in $P_{01}$ |
| UNC-SEV-AMBIG    | ★        | -        | ●        | -        | ●        | ●        | -        | ●        | -        | ●        | ●        | ●        | Discovered in $P_{01}$ |
| HEUR-BLAST-RAD   | ★        | ●        | ●        | -        | ●        | -        | ●        | ●        | ●        | ●        | ●        | ●        | Discovered in $P_{01}$ |
| HEUR-HIST-ANALOG | ★        | ●        | -        | ●        | -        | -        | ●        | ●        | ●        | -        | ●        | ●        | Discovered in $P_{01}$ |
| HEUR-TIME-BOX    | ★        | -        | ●        | -        | ●        | -        | -        | ●        | -        | ●        | ●        | ●        | Discovered in $P_{01}$ |
| HEUR-MIN-DELTA   | ★        | ●        | -        | ●        | -        | ●        | ●        | -        | ●        | ●        | ●        | ●        | Discovered in $P_{01}$ |
| HEUR-PRAG-WORK   | ★        | -        | ●        | [Neg]    | ●        | -        | -        | ●        | -        | ●        | ●        | -        | Discovered in $P_{01}$ |
| ORG-RELEASE-GATE | ★        | ●        | ●        | -        | ●        | -        | ●        | ●        | -        | ●        | ●        | ●        | Discovered in $P_{01}$ |
| ORG-MGMT-ALIGN   | ★        | -        | ●        | -        | -        | ●        | -        | ●        | -        | -        | ●        | ●        | Discovered in $P_{01}$ |
| ORG-LOAD-THROT   | ★        | -        | -        | -        | ●        | -        | ●        | ●        | -        | ●        | ●        | ●        | Discovered in $P_{01}$ |
| ORG-SOC-CONSULT  | -        | ★        | -        | ●        | -        | ●        | -        | ●        | -        | -        | ●        | ●        | Discovered in $P_{02}$ |
| Novel Codes      | 11       | 2        | 0        | 0        | 0        | 0        | 0        | 0        | 0        | 0        | 0        | 0        | Saturation at $P_{08}$ |

Legend: ★ = First empirical emergence of code; ● = Recurrent application of established code; [Neg] = Methodological negative case; - = Code not observed in participant transcript.

**Theoretical Saturation Conclusion:** All 13 core categories emerged within the first two calibration interviews ( $P_{01}, P_{02}$ ) and were comprehensively stabilized across  $P_{03}$  through  $P_{08}$ . Transcripts  $P_{09}$  through  $P_{12}$  contributed rich descriptive corroboration but yielded **zero novel conceptual codes (0%)**, verifying definitive theoretical saturation across the dataset [5].

## References

- [1] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [2] V. Braun and V. Clarke, “Using thematic analysis in psychology,” *Qualitative Research in Psychology*, vol. 3, no. 2, pp. 77–101, 2006.
- [3] Y. S. Lincoln and E. G. Guba, *Naturalistic Inquiry*. SAGE Publications, 1985.
- [4] J. W. Creswell and C. N. Poth, *Qualitative Inquiry and Research Design: Choosing Among Five Approaches*. SAGE Publications, 4th ed., 2018.
- [5] G. Guest, A. Bunce, and L. Johnson, “How many interviews are enough? experimenting with data saturation and variability,” *Field Methods*, vol. 18, no. 1, pp. 59–82, 2006.
- [6] G. A. Klein, *Sources of Power: How People Make Decisions*. MIT Press, 20th anniversary ed. ed., 2017.
- [7] R. Lipshitz, G. Klein, J. Orasanu, and E. Salas, “Taking stock of naturalistic decision making,” *Journal of Behavioral Decision Making*, vol. 14, no. 5, pp. 331–352, 2001.
- [8] D. Bertram, A. Volda, S. Greenberg, and R. Walker, “Communication, collaboration, and bugs: The social nature of issue tracking in small, collocated teams,” in *Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work (CSCW ’10)*, pp. 291–300, ACM, 2010.
- [9] M. Kuuttila, M. M"antyl"a, U. Farooq, and M. Claes, “Time pressure in software engineering: A systematic review,” *Information and Software Technology*, vol. 121, p. 106257, 2020.
- [10] O. Baysal, R. Holmes, and M. W. Godfrey, “No issue left behind: Reducing information overload in issue tracking,” in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE ’14)*, pp. 666–677, ACM, 2014.
- [11] K. A. Ericsson and H. A. Simon, *Protocol Analysis: Verbal Reports as Data*. MIT Press, revised ed., 1993.